

Mockito Hands-On Exercises

Exercise 1: Mocking and Stubbing

ExternalApi.java :

```
public interface ExternalApi {  
    String getData();  
}
```

MyService.java :

```
public class MyService {  
  
    private ExternalApi api;  
  
    public MyService(ExternalApi api) {  
        this.api = api;  
    }  
  
    public String fetchData() {  
        return api.getData();  
    }  
}
```

MyServiceTest.java :

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
import static org.mockito.Mockito.*;  
  
public class MyServiceTest {  
  
    @Test  
    public void testExternalApi() {  
  
        ExternalApi mockApi = mock(ExternalApi.class);  
  
        when(mockApi.getData()).thenReturn("Mock Data");  
  
        MyService service = new MyService(mockApi);  
  
        String result = service.fetchData();  
  
        assertEquals("Mock Data", result);  
    }  
}
```

Maven Dependency (Mockito) :

```
<dependency>  
    <groupId>org.mockito</groupId>  
    <artifactId>mockito-core</artifactId>  
    <version>5.11.0</version>  
    <scope>test</scope>  
</dependency>
```

Output :

Tests run: 1
Failures: 0

Exercise 2: Verifying Interactions

ExternalApi.java :

```
public interface ExternalApi {  
    String getData();  
}
```

MyService.java :

```
public class MyService {  
  
    private ExternalApi api;  
  
    public MyService(ExternalApi api) {  
        this.api = api;  
    }  
  
    public String fetchData() {  
        return api.getData();  
    }  
}
```

MyServiceTest.java :

```
import org.junit.jupiter.api.Test;  
import static org.mockito.Mockito.*;  
  
public class MyServiceTest {  
  
    @Test  
    public void testVerifyInteraction() {  
  
        ExternalApi mockApi = mock(ExternalApi.class);  
  
        MyService service = new MyService(mockApi);  
  
        service.fetchData();  
  
        verify(mockApi).getData();  
    }  
}
```

Output :

Tests run: 1
Failures: 0

Exercise 3: Argument Matching

UserService.java :

```
public interface UserService {  
    void saveUser(String name);  
}
```

UserManager.java :

```
public class UserManager {  
  
    private UserService service;  
  
    public UserManager(UserService service) {  
        this.service = service;  
    }  
  
    public void addUser(String name) {  
        service.saveUser(name);  
    }  
}
```

UserManagerTest.java :

```
import org.junit.jupiter.api.Test;  
import static org.mockito.Mockito.*;  
  
public class UserManagerTest {  
  
    @Test  
    public void testArgumentMatching() {  
  
        UserService mockService = mock(UserService.class);  
  
        UserManager manager = new UserManager(mockService);  
  
        manager.addUser("John");  
  
        verify(mockService).saveUser(eq("John"));  
    }  
}
```

Alternative Using anyString() :

```
verify(mockService).saveUser(anyString());
```

Output :

Tests run: 1
Failures: 0

Exercise 4: Handling Void Methods

Service.java :

```
public interface Service {  
    void performAction();  
}
```

MyManager.java :

```
public class MyManager {  
    private Service service;  
  
    public MyManager(Service service) {  
        this.service = service;  
    }  
  
    public void execute() {  
        service.performAction();  
    }  
}
```

MyManagerTest.java :

```
import org.junit.jupiter.api.Test;  
import static org.mockito.Mockito.*;  
  
public class MyManagerTest {  
    @Test  
    public void testVoidMethod() {  
        Service mockService = mock(Service.class);  
  
        doNothing().when(mockService).performAction();  
  
        MyManager manager = new MyManager(mockService);  
  
        manager.execute();  
  
        verify(mockService).performAction();  
    }  
}
```

Output :

Tests run: 1
Failures: 0

Exercise 5: Mocking and Stubbing with Multiple Returns

ExternalApi.java :

```
public interface ExternalApi {  
    String getData();  
}
```

MyService.java :

```
public class MyService {  
    private ExternalApi api;  
  
    public MyService(ExternalApi api) {  
        this.api = api;  
    }  
}
```

```

    }

    public String fetchData() {
        return api.getData();
    }
}

```

MyServiceTest.java :

```

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class MyServiceTest {

    @Test
    public void testMultipleReturns() {

        ExternalApi mockApi = mock(ExternalApi.class);

        when(mockApi.getData())
            .thenReturn("First Call")
            .thenReturn("Second Call");

        MyService service = new MyService(mockApi);

        assertEquals("First Call", service.fetchData());
        assertEquals("Second Call", service.fetchData());
    }
}

```

Output :

```

Tests run: 1
Failures: 0

```

Exercise 6: Verifying Interaction Order

Service.java :

```

public interface Service {
    void firstMethod();
    void secondMethod();
}

```

MyManager.java :

```

public class MyManager {

    private Service service;

    public MyManager(Service service) {
        this.service = service;
    }

    public void execute() {
        service.firstMethod();
        service.secondMethod();
    }
}

```

```
}  
}
```

MyManagerTest.java :

```
import org.junit.jupiter.api.Test;  
import org.mockito.InOrder;  
  
import static org.mockito.Mockito.*;  
  
public class MyManagerTest {  
  
    @Test  
    public void testInteractionOrder() {  
  
        Service mockService = mock(Service.class);  
  
        MyManager manager = new MyManager(mockService);  
  
        manager.execute();  
  
        InOrder inOrder = inOrder(mockService);  
  
        inOrder.verify(mockService).firstMethod();  
        inOrder.verify(mockService).secondMethod();  
    }  
}
```

Output :

Tests run: 1
Failures: 0

Exercise 7: Handling Void Methods with Exceptions

Service.java :

```
public interface Service {  
    void performAction();  
}
```

MyManager.java :

```
public class MyManager {  
  
    private Service service;  
  
    public MyManager(Service service) {  
        this.service = service;  
    }  
  
    public void execute() {  
        service.performAction();  
    }  
}
```

MyManagerTest.java :

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class MyManagerTest {

    @Test
    public void testVoidMethodException() {

        Service mockService = mock(Service.class);

        doThrow(new RuntimeException("Error"))
            .when(mockService)
            .performAction();

        MyManager manager = new MyManager(mockService);

        assertThrows(
            RuntimeException.class,
            () -> manager.execute()
        );

        verify(mockService).performAction();
    }
}
```

Output :

Tests run: 1
Failures: 0